AI Assistant Coding

Assignment 11.2

Name : N. Eshwar **HT. No :** 2303A52072 **Batch:** 32

Task Description #1 (Refactoring – Removing Code Duplication)

• **Task:** Use AI to refactor a given Python script that contains multiple repeated code blocks.

• **Instructions:**

o Prompt AI to identify duplicate logic and replace it with functions or classes.

o Ensure the refactored code maintains the same output.

o Add docstrings to all functions.

• **Sample Legacy Code:**

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Code:

```
'''
remove duplicates given below code with functions and class with docstrings
'''
def calculate_rectangle_properties(length, width):
    """
    Calculate the area and perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        tuple: A tuple containing the area and perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
```

```
    return area, perimeter

# Example usage
length1, width1 = 5, 10
length2, width2 = 7, 12
length3, width3 = 10, 15

area1, perimeter1 = calculate_rectangle_properties(length1, width1)
area2, perimeter2 = calculate_rectangle_properties(length2, width2)
area3, perimeter3 = calculate_rectangle_properties(length3, width3)

print("Area of Rectangle:", area1)
print("Perimeter of Rectangle:", perimeter1)
print("Area of Rectangle:", area2)
print("Perimeter of Rectangle:", perimeter2)
print("Area of Rectangle:", area3)
print("Perimeter of Rectangle:", perimeter3)
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Output:

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

• Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

• Instructions:

o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

• Sample Legacy Code:

```python
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Code:

```python
'''
refactor the below code to remove the repeated logic and make it more
efficient. Use functions to calculate the total price including tax for
different prices.
'''

# Legacy script with inline repeated logic
def calculate_total_price(price):
    tax = price * 0.18
    total = price + tax
    return total

print("Total Price:", calculate_total_price(250))
print("Total Price:", calculate_total_price(500))
print("Total Price:", calculate_total_price(1000))
```

Output:

```
Total Price: 295.0
Total Price: 590.0
Total Price: 1180.0
```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading

Given code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Code:

```
'''
refactore the code to avoid repetition and make it more efficient
Marks ≥ 90 and ≤ 100 → "Grade A"
▪ Marks ≥ 80 → "Grade B"
▪ Marks ≥ 70 → "Grade C"
▪ Marks ≥ 40 → "Grade D"
▪ Marks ≥ 0 → "Fail"
  with proper classs and methods  with docstring and comments
'''

class Student:
    def __init__(self, marks):
        self.marks = marks
```

```python
    def get_grade(self):
        """
        Get the grade based on the student's marks.

        Returns:
            str: The grade of the student.
        """
        if self.marks >= 90:
            return "Grade A"
        elif self.marks >= 80:
            return "Grade B"
        elif self.marks >= 70:
            return "Grade C"
        elif self.marks >= 40:
            return "Grade D"
        else:
            return "Fail"

# Example usage
student1 = Student(85)
print(student1.get_grade())

student2 = Student(72)
print(student2.get_grade())
```

Output:

```
Grade B
Grade C
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections.

o Convert each into a separate function.

o Improve code readability without changing behavior.

• Sample Legacy Code:

```python
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

Code:

```python
'''
Refactor the code with get_input(), calculate_square(), and
display_result() methods to make it more modular and reusable with proper
docstring and comments
'''

def get_input():
    """
    Get the number from the user.

    Returns:
        int: The number entered by the user.
    """
    return int(input("Enter number: "))

def calculate_square(num):
    """
    Calculate the square of a number.

    Args:
        num (int): The number to square.

    Returns:
        int: The square of the number.
    """
    return num * num

def display_result(square):
    """
    Display the result.

    Args:
        square (int): The square to display.
    """
    print("Square:", square)

# Example usage
num = get_input()
square = calculate_square(num)
display_result(square)
```

Output:

```
Enter number: 5
Square: 25
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design. Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Code:

```
'''
use oops concepts like encapsulation to refactor the code with proper
docstring and comments
with employeesalaryCalculator class and calculate_net_salary() method with
proper methods and attributes like salary and tax working hrs and tax rate
extra hrs working hrs and tax rate
'''
class EmployeeSalaryCalculator:
    """
    A class to calculate the net salary of an employee after tax.
    """

    def __init__(self, salary, tax_rate):
        """
        Initialize the EmployeeSalaryCalculator with salary and tax rate.

        Args:
            salary (float): The gross salary of the employee.
            tax_rate (float): The tax rate to be applied on the salary.
        """
        self.salary = salary
```

```
        self.tax_rate = tax_rate

    def calculate_net_salary(self):
        """
        Calculate the net salary after applying the tax.

        Returns:
            float: The net salary after tax deduction.
        """
        tax = self.salary * self.tax_rate
        net_salary = self.salary - tax
        return net_salary
# Example usage
employee = EmployeeSalaryCalculator(50000, 0.2)
net_salary = employee.calculate_net_salary()
print("Net Salary:", net_salary)
```

Output:

```
40000.0
Net Salary: 40000.0
```

Task 6 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification

Code:

```
'''
Refactor the linear searchs with data structures sets or dictionaries to make
it more efficient with proper docstring and comments with time complexity
analysis

'''
def check_access(username, users):
    """
    Check if the username exists in the users set.

    Args:
        username (str): The username to check.
        users (set): A set of valid usernames.
    Returns:
        bool: True if access is granted, False otherwise.
    """
    return username in users
# Example usage
users_set = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
if check_access(name, users_set):
    print("Access Granted")
else:
    print("Access Denied")
# Time Complexity Analysis:
# The time complexity of checking if an element exists in a set is O(1) on
average, making this approach more efficient than the linear search which has
a time complexity of O(n).
```

Output:

```
Enter username: admin
Access Granted
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe
d:/Course/AIAC/lab13_11_3_2026/task6.py
Enter username: eshwar
Access Denied
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

• Contains repeated conditional logic

• Does not use reusable functions

• Lacks documentation

• Uses print-based procedural execution

• Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:

o add_book(title, author, isbn)

o remove_book(isbn)

o search_book(isbn)

2. Insert triple quotes under each function and let Copilot complete the docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format.

5. Open the file in a browser.

Code:

```
# refactor the above code using class and methods(add_book(title, author,
isbn),remove_book(isbn) ,search_book(isbn))

class Library:

    """
    A simple Library Management System that allows adding, removing, and
    searching for books using their ISBN.
    The library database is stored as a dictionary where the key is the
    ISBN and the value is another dictionary containing the title and author
    of the book.
    """
    def __init__(self):
```

```python
        self.library_db = {}

    def add_book(self, title, author, isbn):
        """
        Add a new book to the library.

        Args:
            title (str): The title of the book
            author (str): The author of the book
            isbn (str): The unique ISBN identifier for the book
        """
        if isbn not in self.library_db:
            self.library_db[isbn] = {"title": title, "author": author}
            print(f"Book '{title}' added successfully.")
        else:
            print(f"Book with ISBN {isbn} already exists.")

    def remove_book(self, isbn):
        """
        Remove a book from the library by its ISBN.

        Args:
            isbn (str): The ISBN identifier of the book to remove
        """
        if isbn in self.library_db:
            book_info = self.library_db[isbn]
            del self.library_db[isbn]
            print(f"Book '{book_info['title']}' removed successfully.")
        else:
            print(f"Book with ISBN {isbn} not found.")

    def search_book(self, isbn):
        """
        Search for a book in the library by its ISBN.

        Args:
            isbn (str): The ISBN identifier of the book to search

        Returns:
            dict or None: Book information if found, None otherwise
        """
        if isbn in self.library_db:
            book = self.library_db[isbn]
            print(f"Book Found: Title='{book['title']}',
Author='{book['author']}', ISBN={isbn}")
            return book
        else:
            print(f"Book with ISBN {isbn} not found.")
```

```python
            return None

    def display_all_books(self):
        """
        Display all books in the library.
        """
        if not self.library_db:
            print("Library is empty.")
        else:
            print(f"\n{'='*60}")
            print(f"{'LIBRARY CATALOG':<60}")
            print(f"{'='*60}")
            for isbn, book in self.library_db.items():
                print(f"ISBN: {isbn} | Title: {book['title']} | Author:
{book['author']}")
            print(f"{'='*60}\n")


# Example Usage
if __name__ == "__main__":
    # Create a library instance
    library = Library()

    print("=== Library Management System Demo ===\n")

    # Adding books
    print("1. Adding books to the library:")
    library.add_book("Python Basics", "John Doe", "101")
    library.add_book("AI Fundamentals", "Jane Smith", "102")
    library.add_book("Data Structures", "Alice Johnson", "103")
    library.add_book("Python Basics", "John Doe", "101")  # Duplicate - should
fail

    print("\n2. Displaying all books:")
    library.display_all_books()

    # Searching for books
    print("3. Searching for books:")
    library.search_book("101")
    library.search_book("999")  # Non-existent book

    # Removing a book
    print("\n4. Removing a book:")
    library.remove_book("101")

    # Searching for removed book
    print("\n5. Searching for the removed book:")
    library.search_book("101")
```

```
    print("\n6. Displaying all books after removal:")
    library.display_all_books()
```

Output:

```
=== Library Management System Demo ===

1. Adding books to the library:
Book 'Python Basics' added successfully.
Book 'AI Fundamentals' added successfully.
Book 'Data Structures' added successfully.
Book with ISBN 101 already exists.

2. Displaying all books:

============================================================
LIBRARY CATALOG
============================================================
ISBN: 101 | Title: Python Basics | Author: John Doe
ISBN: 102 | Title: AI Fundamentals | Author: Jane Smith
ISBN: 103 | Title: Data Structures | Author: Alice Johnson
============================================================

3. Searching for books:
Book Found: Title='Python Basics', Author='John Doe', ISBN=101
Book with ISBN 999 not found.

4. Removing a book:
Book 'Python Basics' removed successfully.

5. Searching for the removed book:
Book with ISBN 101 not found.

6. Displaying all books after removal:

============================================================
LIBRARY CATALOG
============================================================
ISBN: 102 | Title: AI Fundamentals | Author: Jane Smith
ISBN: 103 | Title: Data Structures | Author: Alice Johnson
============================================================
```

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Code:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c
```

Refactor Code:

```
"""Utilities for generating and printing Fibonacci numbers."""


def generate_fibonacci(limit):
    """Return the first ``limit`` Fibonacci numbers as a list.

    Args:
        limit: Number of Fibonacci values to generate.

    Returns:
        A list containing the Fibonacci sequence up to ``limit`` items.

    Raises:
        ValueError: If ``limit`` is negative.
    """
```

```python
    if limit < 0:
        raise ValueError("limit must be non-negative")
    if limit == 0:
        return []
    if limit == 1:
        return [0]

    sequence = [0, 1]
    while len(sequence) < limit:
        sequence.append(sequence[-1] + sequence[-2])
    return sequence


def print_fibonacci(limit):
    """Print the first ``limit`` Fibonacci numbers, one per line."""
    for number in generate_fibonacci(limit):
        print(number)


def main():
    """Read a limit from input and print the Fibonacci sequence."""
    limit = int(input("Enter limit: "))
    print_fibonacci(limit)


if __name__ == "__main__":
    main()
```

Output:

```
Enter limit: 8
0
1
1
2
3
5
8
13
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe
d:/Course/AIAC/lab13_11_3_2026/task8.py
Enter limit: 8
0
1
1
2
3
```

```
5
8
13
Enter limit: 8
0
1
1
2
3
5
8
13
```

Comparison:

Original	Refactor
All logic at global scope	Design with proper function for reusability
Error and edge handling only for 0 ,1 and negative values	It rasies value error if input is wrong and 0, 1
No documentation	With Proper documentation

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

• Inefficient prime checking.

• No functions.

• Hardcoded inputs.

Task for Students:

• Refactor into is_prime(n) and is_twin_prime(p1, p2).

• Add docstrings and optimize.

• Generate a list of twin primes in a given range using AI.

Code:

```
# Refactored and Optimized Twin Primes Checker
```

```python
import math

def is_prime(n):
    """
    Checks if a number is prime in an optimized way.

    Args:
        n (int): The number to check.

    Returns:
        bool: True if the number is prime, False otherwise.
    """
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    # Check only up to the square root of n
    # and step by 6 to skip multiples of 2 and 3.
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def find_twin_primes(start, end):
    """
    Generates a list of twin prime pairs within a given range.

    Args:
        start (int): The beginning of the range.
        end (int): The end of the range.

    Returns:
        list: A list of tuples, where each tuple is a twin prime pair.
    """
    twin_primes = []
    for num in range(start, end - 1):
        if is_prime(num) and is_prime(num + 2):
            twin_primes.append((num, num + 2))
    return twin_primes

def main():
    """
    Main function to demonstrate twin prime checking and generation.
    """
```

```
    print("--- Refactored Twin Primes Checker ---")

    # 1. Check a specific pair of numbers
    p1, p2 = 17, 19
    if is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2:
        print(f"{p1} and {p2} are twin primes.")
    else:
        print(f"{p1} and {p2} are not twin primes.")

    p3, p4 = 23, 29
    if is_prime(p3) and is_prime(p4) and abs(p3 - p4) == 2:
        print(f"{p3} and {p4} are twin primes.")
    else:
        print(f"{p3} and {p4} are not twin primes.")

    # 2. Generate a list of twin primes in a given range
    try:
        start_range = int(input("\nEnter the start of the range: "))
        end_range = int(input("Enter the end of the range: "))

        if start_range >= end_range:
            print("Error: The start of the range must be less than the end.")
        else:
            twin_primes_list = find_twin_primes(start_range, end_range)
            if twin_primes_list:
                print(f"\nTwin primes found between {start_range} and
{end_range}:")
                print(twin_primes_list)
            else:
                print(f"\nNo twin primes found between {start_range} and
{end_range}.")

    except ValueError:
        print("\nInvalid input. Please enter valid integers for the range.")

if __name__ == "__main__":
    main()
```

Output:

```
--- Refactored Twin Primes Checker ---
17 and 19 are twin primes.
23 and 29 are not twin primes.

Enter the start of the range: 10
Enter the end of the range: 500

Twin primes found between 10 and 500:
```

```
[(11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73), (101, 103), (107,
109), (137, 139), (149, 151), (179, 181), (191, 193), (197, 199), (227, 229),
(239, 241), (269, 271), (281, 283), (311, 313), (347, 349), (419, 421), (431,
433), (461, 463)]
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Code:

```python
"""Chinese Zodiac program.

Determines the Chinese Zodiac animal sign for a given year.
"""

ZODIAC_ANIMALS = [
    "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
    "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
]


def get_zodiac(year):
    """Return the Chinese Zodiac animal for the given year.

    Args:
        year (int): The year to look up.

    Returns:
        str: The name of the Chinese Zodiac animal.
    """
    return ZODIAC_ANIMALS[year % 12]


def main():
    """Prompt the user for a year and display the corresponding Zodiac
sign."""
    year = int(input("Enter a year: "))
    print(get_zodiac(year))


if __name__ == "__main__":
    main()
```

Output:

```
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe
d:/Course/AIAC/lab13_11_3_2026/task10.py
Enter a year: 2000
Dragon
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe
d:/Course/AIAC/lab13_11_3_2026/task10.py
Enter a year: 2005
Rooster
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ⍰ (Harshad Number)
- 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ⍰ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

Code:

```python
"""Harshad (Niven) number checker."""


def is_harshad(number: int) -> bool:
    """Return True if number is divisible by the sum of its digits.

    Negative numbers are treated by their absolute value. Zero is not a
    Harshad number because its digit sum is 0.
    """
    if number == 0:
        return False

    remaining = abs(number)
    digit_sum = 0
    while remaining > 0:
        digit_sum += remaining % 10
        remaining //= 10

    return digit_sum != 0 and number % digit_sum == 0
```

```python
def main() -> None:
    """Read input from the user and print the Harshad check result."""
    number = int(input("Enter a number: "))
    print(is_harshad(number))


if __name__ == "__main__":
    main()
```

Output:

```
Enter a number: 18
True
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe
d:/Course/AIAC/lab13_11_3_2026/task11.py
Enter a number: 19
False
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Code:

```python
"""Factorial trailing zeros calculator."""


def count_trailing_zeros(n: int) -> int:
```

```python
    """Return the number of trailing zeros in n!.

    Uses the count of factors of 5. For negative inputs, returns 0.
    """
    if n < 0:
        return 0

    count = 0
    factor = 5
    while n // factor > 0:
        count += n // factor
        factor *= 5

    return count


def main() -> None:
    """Read input from the user and print the trailing zeros count."""
    n = int(input("Enter a number: "))
    print("Trailing zeros:", count_trailing_zeros(n))


if __name__ == "__main__":
    main()
```

Output:

```
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe
d:/Course/AIAC/lab13_11_3_2026/task12.py
Enter a number: 5
Trailing zeros: 1
(AIAC_env) PS D:\Course\AIAC> & D:/Course/AIAC/AIAC_env/Scripts/python.exe
d:/Course/AIAC/lab13_11_3_2026/task12.py
Enter a number: 100
Trailing zeros: 24
(AIAC_env) PS D:\Course\AIAC>
```

Task Description #13: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.

2. Emergency Landing Priority – Aircraft in emergency get priority.

3. Flight Information Lookup – Fast search using Flight ID.

4. Air Route Connectivity – Airports connected via routes.

5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

• Choose suitable data structures from the provided list.

• Provide 2–3 sentence justification per feature.

• Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

• Mapping table.

• Working Python implementation.

Code: